

本物のプログラマーは **unsafe** と **型システム** を使う

拡張可能レコードを作って学ぶ黒魔術入門



Slides & Demo

konn / 2026-07-11 / 関数型まつり2026

<https://u.konn-san.com/fp2026>

自己紹介

いし い ひろ み

- 石井 大海 / konn (Twitter: @mr_konn)
 - 博士（理学）。専門は数理論理学（特に集合論）と計算機代数（Gröbner基底とか）
 - 最近はプログラミング言語理論の周辺にいる気がします
 - 宣伝：EGRAPHS '26 で **e-graph の数理モデラーへの応用**について発表したり、PLDI '26 で 松下祐介氏と **Haskell 内で Rust 流の借用システムを実現する共同研究**を発表したりしました
- 現職：株式会社 Jij ソフトウェア開発チーム（2024/11～）
 - **Rust で数理最適化の埋込ドメイン特化言語 (EDSL) を開発**しています
- 前職では **Haskell** で数値計算向け **EDSL** を開発

本題

本物のプログラマーは

unsafe

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

本物のプログラマーは

講演後の
皆さんのこと

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

unsafe とは？

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
- Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように
 - Haskell の `unsafePerformIO` や `unsafeCoerce` など

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように
 - Haskell の `unsafePerformIO` や `unsafeCoerce` など
 - **IO 処理**をその場で（**純粋な計算として**）実行したり、ある型の値を**別の型にムリヤリ変換**したり

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように
 - Haskell の `unsafePerformIO` や `unsafeCoerce` など
 - **IO 処理**をその場で（**純粋な計算として**）実行したり、ある型の値を**別の型にムリヤリ変換**したり
- 本講演では**もっぱら Haskell について**扱いますが、Rust でも思想は同じ

unsafe にまつわる迷信

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても unsafe があっちゃ意味ないじゃん

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても unsafe があっちゃ意味ないじゃん

😡 unsafe は罪！ unsafe 追放！

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても **unsafe があっちゃ意味ないじゃん**

😡 **unsafe は罪！ unsafe 追放！**

👻 unsafe がエルフの村を焼いたらしい

😞 unsafe はうちの裏には要らない

本当に？

Unsafe な実装が「キケン」なのはいつか
「キケン」にも色々ある

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- ・ そもそも実装しようとしている **API 自体がキケン**なもの

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a -> b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
- API 自体はアンゼンだが**実装にミスがある**場合

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全だが……

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
 - API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全だが……
- ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
 - API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 掛けちゃだめ！
- ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 掛けちゃだめ！
 - ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a -> b` があると任意の型の値が手に入る！
(unsafeCoerce の **unsafe たるゆえん**)
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] -> Int` はそれ型それ自体は安全

掛けちゃだめ！

① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`

② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

空リストで実行時エラー！

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性** (**correctness**) : プログラムが仕様を本当に満たしているか？

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど
- ・ **健全性 (soundness)** : 今考えているAPIが天から与えられたとして、**「型システムなどの言語処理系が当然保証すべき安全性」**が破れないか？

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど
- ・ **健全性 (soundness)** : 今考えているAPIが天から与えられたとして、**「型システムなどの言語処理系が当然保証すべき安全性」**が破れないか？
 - ・ 型システムが念頭にある場合は**型安全性 (type safety)** とも言う

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど
- ・ **健全性 (soundness)** : 今考えているAPIが天から与えられたとして、**「型システムなどの言語処理系が当然保証すべき安全性」**が破れないか？
 - ・ 型システムが念頭にある場合は**型安全性 (type safety)** とも言う
 - ・ **暗黙に**「保証されてほしい性質」が仮定されており、**文脈依存の概念**

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- ・ 「キケン」性を測る尺度をここでは**二つ**考える
- ・ **正当性 (correctness)** : プログラムが仕様を本当に満たしているか？
 - ・ `sum :: [Int] → Int` なのに積を取っているなど
- ・ **健全性 (soundness)** : 今考えているAPIが天から与えられたとして、**「型システムなどの言語処理系が当然保証すべき安全性」**が破れないか？
 - ・ 型システムが念頭にある場合は**型安全性 (type safety)** とも言う
 - ・ **暗黙に**「保証されてほしい性質」が仮定されており、**文脈依存の概念**
 - ・ `unsafeCoerce :: a → b` や立場によっては（厳密には）`head :: [a] → a` もそう

「unsafe は「キケン」なので良くない」

- head や tail、配列へのランダムアクセスだって尺度により「**キケン**」
- でも「そこは保証しない」という**デザインチョイス**になっている
 - 極力避けたいのでHaskell では非空性が保証された NonEmpty というリストの亜種を可能な限り使うことが最近では一般化しつつある
- 良いことは「何がキケン」なのかわからないこと

どう「unsafe」を使っていくべきか？

どう「unsafe」を使っていくべきか？

- ・ 使わずに済むなら使わないに越したことはない

どう「unsafe」を使っていくべきか？

- ・ 使わずに済むなら使わないに越したことはない
- ・ 一方で、用法・用量を心得て使えば安全性と効率性を両立したライブラリを創れるようになる

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？
- ・ 今回の設定：拡張可能レコード

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？
- ・ 今回の設定：拡張可能レコード
 - ・ まず**安全だが遅い実装**をしてAPI健全性を確認

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？
- ・ 今回の設定：拡張可能レコード
 - ・ まず**安全だが遅い実装**をしてAPI健全性を確認
 - ・ Haskell の**強力な型レベルプログラミングの入門**もします

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？
- ・ 今回の設定：拡張可能レコード
 - ・ まず**安全だが遅い実装**をしてAPI健全性を確認
 - ・ Haskell の**強力な型レベルプログラミングの入門**もします
 - ・ その後 **unsafe な処理を使った高速な実装**で差し替え

どう「unsafe」を使っていくべきか？

- ・ **使わずに済むなら使わないに越したことはない**
- ・ 一方で、**用法・用量を心得て**使えば**安全性と効率性を両立**したライブラリを創れるようになる
- ・ どのように付き合っていく？
- ・ 今回の設定：拡張可能レコード
 - ・ まず**安全だが遅い実装**をしてAPI健全性を確認
 - ・ Haskell の**強力な型レベルプログラミングの入門**もします
 - ・ その後 **unsafe な処理を使った高速な実装**で差し替え
 - ・ おまけ：線型型による効率と安全性の「先」

Case Study: 拡張可能レコード



Implementation

<https://u.konn-san.com/fp2026>

Case Study: 拡張可能レコード

Case Study: 拡張可能レコード

- ・ レコード：フィールドと値の束

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する
- 拡張可能レコード（匿名レコードとも）：フィールドと型の組み合わせを自由に増減させたり、フィールドの非／存在に関する多相関数を書いたりできる

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する
- 拡張可能レコード（匿名レコードとも）：フィールドと型の組み合わせを自由に増減させたり、フィールドの非／存在に関する多相関数を書いたりできる
- 型安全なプラグイン機構やイフェクトシステムの実装に使える

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する
- 拡張可能レコード（匿名レコードとも）：フィールドと型の組み合わせを自由に増減させたり、フィールドの非／存在に関する多相関数を書いたりできる
- 型安全なプラグイン機構やイフェクトシステムの実装に使える
 - 前職では拡張可能レコードをベースとした型安全なプラグインシステムを実用していました

レコードの例

点を表すレコード Point と円を表す Circle を定義

```
data Point = Point {x :: Double, y :: Double}

data Circle = Circle
  { x :: Double
  , y :: Double
  , radius :: Double
  }
```

拡張可能レコード例：一部のフィールドだけに興味がある

拡張可能レコード例：一部のフィールドだけに興味がある

- Circle, Point の平行移動を書こうと思うと、個別に書く必要がある

```
movePoint :: Double → Double → Point → Point
movePoint dx dy pt = pt {x = pt.x + dx, y = pt.y + dy}

moveCircle :: Double → Double → Circle → Circle
moveCircle dx dy c = c {x = c.x + dx, y = c.y + dy}
```

拡張可能レコード例：一部のフィールドだけに興味がある

- Circle, Point の平行移動を書こうと思うと、個別に書く必要がある

OverloadedRecordDot

```
movePoint :: Double → Double → Point → Point
movePoint dx dy pt = pt {x = pt.x + dx, y = pt.y + dy}

moveCircle :: Double → Double → Circle → Circle
moveCircle dx dy c = c {x = c.x + dx, y = c.y + dy}
```

拡張可能レコード例：一部のフィールドだけに興味がある

- Circle, Point の平行移動を書こうと思うと、個別に書く必要がある

OverloadedRecordDot

```
movePoint :: Double → Double → Point → Point
movePoint dx dy pt = pt {x = pt.x + dx, y = pt.y + dy}

moveCircle :: Double → Double → Circle → Circle
moveCircle dx dy c = c {x = c.x + dx, y = c.y + dy}
```

- 以下のように、特定のフィールドの有無だけ考えて書けたらなあ……

```
move :: (("x" := Double) ∈ fs, ("y" := Double) ∈ fs) ⇒
       Double → Double → Record fs → Record fs
move dx dy r = r {x = r.x + dx, y = r.y + dy}
```

- 他にも、フィールドを後から増やせたりしたら便利そう

どうやって実現する？

- Haskell では**型レベルに文字列・リストを持ち上げる**ことができる
- `{ x :: Double, y :: Double, name :: String }` は次のように書けそう
 - `Record '["x" := Double, "y" := Double, "name" := String]`
- 具体的にどういうインタフェースがあればいいだろう？

満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type
data Record fs

empty :: Record '[]

type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs

getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs

type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs

insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type
data Record fs
```

型レベル文字列

```
empty :: Record '[]
```

```
type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs
```

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs
```

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```


満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type  
data Record fs
```

型レベル文字列

```
empty :: Record '[]
```

型制約

```
type (∈) :: Symbol → [(Symbol, Type)] → Constraint  
class l ∈ fs
```

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
```

```
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint  
class l ∉ fs
```

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type
data Record fs
```

型レベル文字列

```
empty :: Record '[]
```

型制約

```
type (∈) :: Symbol → Constraint
class l ∈ fs
```

RequiredTypeArguments: Constraint
型を引数として取れる

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs
```

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```


満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type
data Record fs
```

型レベル文字列

```
empty :: Record '[]
```

型制約

```
type (∈) :: Symbol → Type → Constraint
class l ∈ fs
```

RequiredTypeArguments: 型を引数として取れる

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs
```

'(l, v) と書くのがダルいので略記

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

満たすべきAPI

```
type Record :: [(Symbol, Type)] → Type
data Record fs
```

型レベル文字列

```
empty :: Record '[]
```

型制約

```
type (∈) :: Symbol → Type → Constraint
class l ∈ fs
```

RequiredTypeArguments: 型を引数として取れる

型レベルの
lookup + fromJust

```
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Lookup' l fs → Record fs → Lookup' l fs
```

```
type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs
```

'(l, v) と書くのがダルいので略記

```
insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record (l := v : fs)
```

型族による型レベル関数（みたいなもの）

```
type Lookup :: Symbol → [(Symbol, v)] → Maybe v
type family Lookup l kvs where
  Lookup l '[] = 'Nothing
  Lookup l (l  := v ': fs) = 'Just v
  Lookup l (l' := v ': fs) = Lookup l fs

type Lookup' :: Symbol → [(Symbol, v)] → v
type Lookup' l kvs =
  FromJust
    (ShowType l :◇: Text " not found in fields " :◇: ShowType kvs)
    (Lookup l kvs)

type family FromJust e may where
  FromJust e ('Just v) = v
  FromJust e 'Nothing = TypeError e
```


型族による型レベル関数（みたいなもの）

```
type Lookup :: Symbol → [(Symbol, v)] → Maybe v
type family Lookup l kvs where
  Lookup l '[] = 'Nothing
  Lookup l (l  := v ': fs) = 'Just v
  Lookup l (l' := v ': fs) = Lookup l fs
```

```
type Lookup' :: Symbol → [(Symbol, v)] → v
type Lookup' l kvs =
  FromJust
```

```
    (ShowType l :◇: Text " not found in fields " :◇: ShowType kvs)
    (Lookup l kvs)
```

Nothing の場合のエラーメッセージ

```
type family FromJust e may where
  FromJust e ('Just v) = v
  FromJust e 'Nothing = TypeError e
```

型族による型レベル関数（みたいなもの）

```
type Lookup :: Symbol → [(Symbol, v)] → Maybe v
type family Lookup l kvs where
  Lookup l '[] = 'Nothing
  Lookup l (l  := v ': fs) = 'Just v
  Lookup l (l' := v ': fs) = Lookup l fs
```

```
type Lookup' :: Symbol → [(Symbol, v)] → v
type Lookup' l kvs =
  FromJust
```

```
  (ShowType l :◇: Text " not found in fields " :◇: ShowType kvs)
  (Lookup l kvs)
```

```
type family FromJust e may where
  FromJust e ('Just v) = v
```

```
  FromJust e 'Nothing = TypeError e
```

Nothing の場合のエラーメッセージ

Nothing なのでエラー発生！

このAPI、
健全？

実装して確認！

Step 1: 型システムを使い倒して 安全に実装する



Implementation
<https://u.konn-san.com/fp2026>

実装法：API の健全性を確かめる簡単な方法

実装法：API の健全性を確かめる簡単な方法

- ・ 効率を度外視して、**「安全」な言語機能だけ**を使って API を実装してみる

実装法：API の健全性を確かめる簡単な方法

- ・ 効率を度外視して、**「安全」な言語機能だけ**を使って API を実装してみる
 - ・ **undefined や unsafe 関数などは封印……**

実装法：API の健全性を確かめる簡単な方法

- ・ 効率を度外視して、**「安全」な言語機能だけ**を使って API を実装してみる
 - ・ **undefined や unsafe 関数などは封印……**
 - ・ head とかランダムアクセスも**可能な限り避ける**

実装法：API の健全性を確かめる簡単な方法

- 効率を度外視して、**「安全」な言語機能だけ**を使って API を実装してみる
 - **undefined や unsafe 関数などは封印……**
 - head とかランダムアクセスも**可能な限り避ける**
- これができれば、**「この API を足しても気を付けて書いた Haskell と同程度には健全」**という事が確かめられる

実装法：API の健全性を確かめる簡単な方法

- 効率を度外視して、「安全」な言語機能だけを使って API を実装してみる
 - `undefined` や `unsafe` 関数などは封印……
 - `head` とかランダムアクセスも可能な限り避ける
- これができれば、「この API を足しても気を付けて書いた Haskell と同程度には健全」という事が確かめられる
 - 数理論理的には「定義による拡張」を試みることに対応

そんなこと出来るの？

A. GADTs[※]

使えば[”]できる！

※ Rust の GATs (Generalised Associated Types) とは異なるものです

GADTs: 一般化代数的データ型

- ・ パターンマッチによってパラメータの型を確定させられるデータ型

GADTs: 一般化代数的データ型

- ・ パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```


GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```

GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```

GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```

$a \sim \text{Int}$

$a \sim \text{Bool}$

$a \sim u1 \rightarrow u2$

GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```

GADTs: 一般化代数的データ型

- パターンマッチによってパラメータの型を確定させられるデータ型

```
data U x where
  IsInt  :: U Int
  IsBool :: U Bool
  IsPair :: U a → U b → U (a, b)
  IsFun  :: U a → U b → U (a → b)
```

興味がある型の宇宙

```
genU :: U a → a
genU IsInt = 0
genU IsBool = False
genU (IsPair u1 u2) = (genU u1, genU u2)
genU (IsFun u1 u2) = \_ → genU u2
```

$a \sim u1 \rightarrow u2$

$a \sim u1 \rightarrow u2$

$a \sim \text{Bool}$

$a \sim \text{Int}$

```
data V a n where
  Nil  :: V a 0
  (:>) :: a → V a n → V a (n + 1)
```

長さつきベクトル

GADTs の応用例：異種リスト

- 異種リスト（Heterogeneous List、略してHList）は GADTs の主要な応用の一つ
- 成分ごとに型がちがう Cons リストが書ける

```
data HList xs where
  HNil :: HList '[]
  (:-) :: x → HList xs → HList (x : xs)
```

```
h1 :: HList '[Int, String, Bool]
h1 = 12 :- "foo" :- False :- HNil
```

```
htail :: HList (x : xs) → HList xs
htail (_ :- xs) = xs
```

```
hhead :: HList (x : xs) → x
hhead (x :- _) = x
```

GADTs の応用例：異種リスト

- 異種リスト（Heterogeneous List、略してHList）は GADTs の主要な応用の一つ
- 成分ごとに型がちがう Cons リストが書ける

```
data HList xs where
  HNil :: HList '[]
  (:-) :: x → HList xs → HList (x : xs)
```

```
h1 :: HList '[Int, String, Bool]
h1 = 12 :- "foo" :- False :- HNil
```

```
htail :: HList (x : xs) → HList xs
htail (_ :- xs) = xs
```

```
hhead :: HList (x : xs) → x
hhead (x :- _) = x
```

空の場合
コンパイル不可

実は HList が
拡張可能レコードの
実装に重要！

HList 風 「レコード」 1

HList 風「レコード」1

- HList とほぼ同じで、ラベルの情報だけ余分につけてみる

HList 風「レコード」1

- HList とほぼ同じで、ラベルの情報だけ余分につけてみる

```
data Record fs where  
  NilR :: Record '[]  
  (:>) :: v → Record fs → Record (l := v : fs)
```


HList 風「レコード」1

- HList とほぼ同じで、ラベルの情報だけ余分につけてみる

```
data Record fs where
  NilR :: Record '[]
  (:>) :: v → Record fs → Record (l := v : fs)
```

- 次は $\in$ と getField を考えたい：

HList 風「レコード」1

- HList とほぼ同じで、ラベルの情報だけ余分につけてみる

```
data Record fs where  
  NilR :: Record '[]  
  (:>) :: v → Record fs → Record (l := v : fs)
```

- 次は $\in$ と getField を考えたい：

```
class l ∈ fs  
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
```


HList 風「レコード」1

- HList とほぼ同じで、ラベルの情報だけ余分につけてみる

```
data Record fs where
  NilR :: Record '[]
  (:>) :: v → Record fs → Record (l := v : fs)
```

- 次は $\in$ と getField を考えたい：

```
class l ∈ fs
getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
```

💡 Record は実質 Cons リストなので、**頭からマッチ**したらどうか？

HList 風「レコード」 2

```
class l ∈ fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs)

instance {-# OVERLAPPABLE #-} (l ∈ fs)
  ⇒ l ∈ (kv : fs)
```

HList 風「レコード」 2

```
class l ∈ fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs)

instance {-# OVERLAPPABLE #-} (l ∈ fs)
  ⇒ l ∈ (kv : fs)
```

- とりあえず、フィールドについて再帰してみる

HList 風「レコード」 2

```
class l ∈ fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs)

instance {-# OVERLAPPABLE #-} (l ∈ fs)
  ⇒ l ∈ (kv : fs)
```

- とりあえず、フィールドについて再帰してみる
 - OVERLAPPING, OVERLAPPABLE: 型クラスは関数と違って定義順の影響を受けないので、結果が被り得る場合どっちを優先するのか決める必要がある

HList 風「レコード」 2

```
class l ∈ fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs)

instance {-# OVERLAPPABLE #-} (l ∈ fs)
  ⇒ l ∈ (kv : fs)
```

- とりあえず、フィールドについて再帰してみる
 - OVERLAPPING, OVERLAPPABLE: 型クラスは関数と違って定義順の影響を受けないので、結果が被り得る場合どちらを優先するのか決める必要がある

🤔 getField と update はどう書けばいい？

メンバ関数にしてみる

```
class l ∈ fs where
  getField :: Proxy l → Record fs → Lookup' l fs
  update   :: Proxy l → Lookup' l fs → Record fs → Record fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs) where
  getField l (x :> _) = x
  update l x (_ :> xs) = x :> xs

instance {-# OVERLAPPABLE #-} (l ∈ fs) ⇒ l ∈ (kv : fs) where
  getField l (_ :> xs) = getField l xs
  update l x (y :> xs) = y :> update l x xs
```

- これでいけそうだが……

エラーになる

エラーになる

```
instance {-# OVERLAPPABLE #-} (l ∈ fs) ⇒ l ∈ (kv : fs) where
  getField l (_ :> xs) = getField l xs
  update l x (y :> xs) = y :> update l x xs
```

- Could not deduce
 'FromJust
 ((ShowType l :◇: Text " not found in fields ")
 :◇: ShowType ((l1 := v) : fs))
 (Lookup l ((l1 := v) : fs))
 ~ FromJust
 ((ShowType l :◇: Text " not found in fields ") :◇: ShowType fs)
 (Lookup l fs)'
 from the context: (kv : fs) ~ ((l1 := v) : fs1)

エラーになる

```
instance {-# OVERLAPPABLE #-} (l ∈ fs) ⇒ l ∈ (kv : fs) where
  getField l (_ :> xs) = getField l xs
  update l x (y :> xs) = y :> update l x xs
```

- Could not deduce
 'FromJust
 ((ShowType l :◇: Text " not found in fields ")
 :◇: ShowType ((l1 := v) : fs))
 (Lookup l ((l1 := v) : fs))
 ~ FromJust
 ((ShowType l :◇: Text " not found in fields ") :◇: ShowType fs)
 (Lookup l fs)'
 from the context: (kv : fs) ~ ((l1 := v) : fs1)

- l が一致する／しないの条件がなく、Lookup の定義とパスが一致していないため

制約に足してしまおう

```
instance {-# OVERLAPPABLE #-}  
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)  
  ⇒ l ∈ (kv : fs) where  
  getField l (_ :> xs) = getField l xs  
  update l x (y :> xs) = y :> update l x xs
```

制約に足してしまおう

```
instance {-# OVERLAPPABLE #-}  
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)  
  ⇒ l ∈ (kv : fs) where  
  getField l (_ :> xs) = getField l xs  
  update l x (y :> xs) = y :> update l x xs
```

- ・ 先頭が一致していれば OVERLAPPING が優先され、こちらはフォールバックになる

制約に足してしまおう

```
instance {-# OVERLAPPABLE #-}  
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)  
  ⇒ l ∈ (kv : fs) where  
  getField l (_ :> xs) = getField l xs  
  update l x (y :> xs) = y :> update l x xs
```

- 先頭が一致していれば OVERLAPPING が優先され、こちらはフォールバックになる
- むしろ最初から以下のように Lookup だけで $\in$ を定義できないのか？

制約に足してしまおう

```
instance {-# OVERLAPPABLE #-}  
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)  
  ⇒ l ∈ (kv : fs) where  
  getField l (_ :> xs) = getField l xs  
  update l x (y :> xs) = y :> update l x xs
```

- 先頭が一致していれば OVERLAPPING が優先され、こちらはフォールバックになる
- むしろ最初から以下のように Lookup だけで $\in$ を定義できないのか？

```
type l ∈ fs = Lookup l fs ~ 'Just (Lookup' l fs)
```

制約に足してしまおう

```
instance {-# OVERLAPPABLE #-}  
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)  
  ⇒ l ∈ (kv : fs) where  
  getField l (_ :> xs) = getField l xs  
  update l x (y :> xs) = y :> update l x xs
```

- ・ 先頭が一致していれば OVERLAPPING が優先され、こちらはフォールバックになる
- ・ むしろ最初から以下のように Lookup だけで $\in$ を定義できないのか？

```
type l ∈ fs = Lookup l fs ~ 'Just (Lookup' l fs)
```

➡ getField / update **が触るべき位置**がわからないのでダメ！

構成的に考える：「位置」を表す証拠

構成的に考える：「位置」を表す証拠

- ・ 「リスト xs のどこの要素か？」を表す Membership x xs という型を考える

構成的に考える：「位置」を表す証拠

- ・ 「リスト xs のどここの要素か？」を表す `Membership x xs` という型を考える

```
data Membership :: k → [k] → Type where
  Here  :: Membership l (l : fs)
  There :: Membership l fs → Membership l (l' : fs)

mem1 = There (There Here) :: Membership 2 '[0, 1, 2, 3, 4]
```

構成的に考える：「位置」を表す証拠

- ・ 「リスト xs のどこの要素か？」を表す `Membership x xs` という型を考える

```
data Membership :: k -> [k] -> Type where
  Here  :: Membership l (l : fs)
  There :: Membership l fs -> Membership l (l' : fs)

mem1 = There (There Here) :: Membership 2 '[0, 1, 2, 3, 4]
```

- ・ これがあれば、`getField` , `update` を書くには十分：

構成的に考える：「位置」を表す証拠

- ・ 「リスト xs のどこの要素か？」を表す `Membership x xs` という型を考える

```
data Membership :: k -> [k] -> Type where
  Here  :: Membership l (l : fs)
  There :: Membership l fs -> Membership l (l' : fs)

mem1 = There (There Here) :: Membership 2 '[0, 1, 2, 3, 4]
```

- ・ これがあれば、`getField` , `update` を書くには十分：

```
getFieldOf :: Membership (l := v) fs -> Record fs -> v
getFieldOf Here (v :> _) = v
getFieldOf (There m) (_ :> fs) = getFieldOf m fs

updateOf :: Membership (l := v) fs -> v -> Record fs -> Record fs
updateOf Here v (_ :> fs) = v :> fs
updateOf (There m) v (x :> fs) = x :> updateOf m v fs
```


構成的に考える：「位置」を表す証拠

- ・ 「リスト xs のどこの要素か？」を表す `Membership x xs` という型を考える

```
data Membership :: k → [k] → Type where
  Here  :: Membership l (l : fs)
  There :: Membership l fs → Membership l (l' : fs)

mem1 = There (There Here) :: Membership 2 '[0, 1, 2, 3, 4]
```

- ・ これがあれば、`getField` , `update` を書くには十分：

```
getFieldOf :: Membership (l := v) fs → Record fs → v
getFieldOf Here (v := _) = v
getFieldOf (There m) (_ := fs) = getFieldOf m fs

updateOf :: Membership (l := v) fs → v → Record fs → Record fs
updateOf Here v (_ := fs) = v := fs
updateOf (There m) v (x := fs) = x := updateOf m v fs
```

➡逆に `Membership` を基に `x ∈ xs` を再定義できないか？

Membership による $\in$ の再定義

Membership による $\in$ の再定義

- $l \in xs$ をその**証拠**を返してくれる単機能のクラスにする

Membership による $\in$ の再定義

- $l \in xs$ をその**証拠**を返してくれる単機能のクラスにする

```
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs) where
  membership = Here

instance {-# OVERLAPPABLE #-}
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)
  ⇒ l ∈ (kv : fs) where
  membership = There membership

getField l = getFieldOf (membership @l @fs)

update l = updateOf (membership @l @fs)
```

Membership による $\in$ の再定義

- $l \in xs$ をその**証拠**を返してくれる単機能のクラスにする
- 残りの演算は Membership の持っている証拠を使って具体的に構成できる！

```
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs) where
  membership = Here

instance {-# OVERLAPPABLE #-}
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)
  ⇒ l ∈ (kv : fs) where
  membership = There membership

getField l = getFieldOf (membership @l @fs)

update l = updateOf (membership @l @fs)
```


Membership による $\in$ の再定義

- $l \in xs$ をその**証拠**を返してくれる単機能のクラスにする
- 残りの演算は Membership の持っている証拠を使って具体的に構成できる！

🤔 綺麗だけど、現時点で意味ある？

```
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs) where
  membership = Here

instance {-# OVERLAPPABLE #-}
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)
  ⇒ l ∈ (kv : fs) where
  membership = There membership

getField l = getFieldOf (membership @l @fs)

update l = updateOf (membership @l @fs)
```

Membership による $\in$ の再定義

- $l \in xs$ をその**証拠**を返してくれる単機能のクラスにする
 - 残りの演算は Membership の持っている証拠を使って具体的に構成できる！
- 🤔 綺麗だけど、現時点で意味ある？
- unsafe を使うときに効いてきます！

```
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs

instance {-# OVERLAPPING #-} l ∈ (l := v : fs) where
  membership = Here

instance {-# OVERLAPPABLE #-}
  (l ∈ fs, Lookup' l (kv : fs) ~ Lookup' l fs)
  ⇒ l ∈ (kv : fs) where
  membership = There membership

getField l = getFieldOf (membership @l @fs)

update l = updateOf (membership @l @fs)
```

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- ・ **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する
- **直観主義論理**：排中律 $A \vee \neg A$ を認めない論理。「構成的」な数学を行うための枠組みと思える

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する
- **直観主義論理**：排中律 $A \vee \neg A$ を認めない論理。「構成的」な数学を行うための枠組みと思える
 - **BHK 解釈**：C-H対応の先駆*。直観主義論理の証明は「**具体的な証明の構成法**」であると捉える

※Howard はKreiselの影響下でCH対応を定式化していて、提唱した当時は BHK のことはあまりよく知らなかったらしい

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する
- **直観主義論理**：排中律 $A \vee \neg A$ を認めない論理。「構成的」な数学を行うための枠組みと思える
- **BHK 解釈**：C-H対応の先駆*。直観主義論理の証明は「**具体的な証明の構成法**」であると捉える
 - 「 $A \rightarrow B$ 」は「 A の証明から B の証明の "構成" 法」、「 $A \vee B$ 」は「 A か B どちらかの証明」、「 $A \wedge B$ 」は「 A の証明と B の証明の組」などなど

※Howard はKreiselの影響下でCH対応を定式化していて、提唱した当時は BHK のことはあまりよく知らなかったらしい

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する
- **直観主義論理**：排中律 $A \vee \neg A$ を認めない論理。「構成的」な数学を行うための枠組みと思える
 - **BHK 解釈**：C-H対応の先駆*。直観主義論理の証明は「**具体的な証明の構成法**」であると捉える
 - 「 $A \rightarrow B$ 」は「A の証明から B の証明の "構成" 法」、「 $A \vee B$ 」は「A か B どちらかの証明」、「 $A \wedge B$ 」は「A の証明と B の証明の組」などなど
- 型レベルプログラミングではこの考え方を逆用して、Membership $x \ xs$ を用いた $\in$ クラスの定式化のように何らかの**情報を含んだ「証拠」を基盤に置く**とうまくいきやすい

※Howard はKreiselの影響下でCH対応を定式化していて、提唱した当時は BHK のことはあまりよく知らなかったらしい

余談：構成的数学と型レベルプログラミング

Brouwer–Heyting–Kolmogorov 解釈を悪用する

- **Curry–Howard 対応**：直観主義論理と λ -計算の間の対応。主要な**定理証明系の理論的基礎**の一つ
 - 簡単にいうと、**型**は**命題**に、**証明**は**プログラム**に対応する
- **直観主義論理**：排中律 $A \vee \neg A$ を認めない論理。「構成的」な数学を行うための枠組みと思える
 - **BHK 解釈**：C-H対応の先駆*。直観主義論理の証明は「**具体的な証明の構成法**」であると捉える
 - 「 $A \rightarrow B$ 」は「 A の証明から B の証明の "構成" 法」、「 $A \vee B$ 」は「 A か B どちらかの証明」、「 $A \wedge B$ 」は「 A の証明と B の証明の組」などなど
- 型レベルプログラミングではこの考え方を逆用して、Membership $x \ xs$ を用いた $\in$ クラスの定式化のように何らかの**情報を含んだ「証拠」を基盤に置く**とうまくいきやすい
 - **何が「証拠」として相応しいか？**を考えると自ずと問題が解けることも

※Howard はKreiselの影響下でCH対応を定式化していて、提唱した当時は BHK のことはあまりよく知らなかったらしい

insert と $\notin$

```
type l  $\notin$  fs = Lookup l fs ~ 'Nothing

insert ::
   $\forall$  fs v.  $\forall$  l  $\rightarrow$  (l  $\notin$  fs)  $\Rightarrow$ 
  v  $\rightarrow$  Record fs  $\rightarrow$  Record (l := v : fs)
insert _ x fs = x :> fs
```

- 「無い」ことがわかっていれば cons するだけなので Lookup だけで書ける

API はすべて Safe に実装できた！

```
type Record :: [(Symbol, Type)] → Type
data Record fs

type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs

getField :: ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs
update  :: ∀ v fs. ∀ l → (l ∈ fs) ⇒ Record fs → Lookup' l fs

type (∉) :: Symbol → [(Symbol, Type)] → Constraint
class l ∉ fs

insert :: ∀ v fs. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

- GADTs による HList + Membership による所属関係の証拠の表現

まとめ

- **GADTs と型族**を使うと、効率を無視すれば**意外と色々な API を実装**できる
 - もちろん、出来ない場合もある
 - **GADTs = パターンマッチで型パラメータを確定**させられる ADT
 - **型族** = 型レベルの関数（みたいなもの）
 - 拡張可能レコードを**要素ごとに型の異なる異種Consリスト**として実装できた
- 型レベルプログラミングでは表現したい性質の「**証拠**」を意識して型制約を設計すべし！
 - $l \in ls$ は単に入っていることを制約に表すだけでなく、具体的にその位置の情報を取り出せる「**証拠**」 `Membership l ls` を使って定式化した

Step 2: unsafe 実装による
高速化

これまでの実装の問題点

ランダムアクセスが遅い

これまでの実装の問題点

ランダムアクセスが遅い

🤔 API が実装できたのなら、わざわざ unsafe 使わなくてもいいのでは？

これまでの実装の問題点

ランダムアクセスが遅い

🤔 API が実装できたのなら、わざわざ unsafe 使わなくてもいいのでは？

💣 フィールドアクセスが $O(N)$ になっちゃう！

これまでの実装の問題点

ランダムアクセスが遅い

🤔 API が実装できたのなら、わざわざ unsafe 使わなくてもいいのでは？

💣 フィールドアクセスが $O(N)$ になっちゃう！

- `Record '["f1" := A1, .., "f100" := A100]` から `f100` の値を読むには、
99個の Cons を剥がさないとイケない！

これまでの実装の問題点

ランダムアクセスが遅い

🤔 API が実装できたのなら、わざわざ unsafe 使わなくてもいいのでは？

💣 フィールドアクセスが $O(N)$ になっちゃう！

- `Record '["f1" := A1, .., "f100" := A100]` から `f100` の値を読むには、
99個の Cons を剥がさないとイケない！

🤔 一般的なレコードと同様に $O(1)$ を期待したい。どうすれば……

これまでの実装の問題点

ランダムアクセスが遅い

🤔 API が実装できたのなら、わざわざ unsafe 使わなくてもいいのでは？

💣 **フィールドアクセスが $O(N)$ になっちゃう！**

- `Record '["f1" := A1, .., "f100" := A100]` から `f100` の値を読むには、**99個の Cons** を剥がさないとイケない！

🤔 一般的なレコードと同様に $O(1)$ を期待したい。どうすれば……

💡 **配列／ベクトル**を使えば $O(1)$ のランダムアクセスが出来るのでは！？

黒魔術で異種ベクトルを「作る」

黒魔術で異種ベクトルを「作る」

- ・ 我々は Record を要素ごとに型が異なるリストとして定式化した

黒魔術で異種ベクトルを「作る」

- ・ 我々は Record を要素ごとに型が異なるリストとして定式化した
- ・ Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定

黒魔術で異種ベクトルを「作る」

- ・ 我々は Record を要素ごとに型が異なるリストとして定式化した
- ・ Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定
- ・ ここで黒魔術の登場：

黒魔術で異種ベクトルを「作る」

- ・ 我々は Record を要素ごとに型が異なるリストとして定式化した
- ・ Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定
- ・ ここで黒魔術の登場：
 1. `unsafeCoerce :: a → b` 任意の値を任意の型に変換できる黒魔術

黒魔術で異種ベクトルを「作る」

- ・ 我々は Record を要素ごとに型が異なるリストとして定式化した
- ・ Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定
- ・ ここで黒魔術の登場：
 1. `unsafeCoerce :: a -> b` 任意の値を任意の型に変換できる黒魔術
 2. `Any` 型：`unsafeCoerce` との「可換性」が保証されている型 (`GHC.Exts`)

黒魔術で異種ベクトルを「作る」

- 我々は Record を要素ごとに型が異なるリストとして定式化した
- Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定
- ここで黒魔術の登場：
 1. `unsafeCoerce :: a → b` 任意の値を任意の型に変換できる黒魔術
 2. `Any` 型：`unsafeCoerce` との「可換性」が保証されている型 (`GHC.Exts`)
 - `a = unsafeCoerce (unsafeCoerce (a :: T) :: Any) :: T` が保証されている

黒魔術で異種ベクトルを「作る」

- 我々は Record を要素ごとに型が異なるリストとして定式化した
- Haskell の配列・ベクトルは `Vector a` のように要素は単一の型 `a` に固定
- ここで黒魔術の登場：
 1. `unsafeCoerce :: a → b` 任意の値を任意の型に変換できる黒魔術
 2. `Any` 型：`unsafeCoerce` との「可換性」が保証されている型 (`GHC.Exts`)
 - `a = unsafeCoerce (unsafeCoerce (a :: T) :: Any) :: T` が保証されている
- 戦略：`Vector Any` として実装し、要素を出し入れする時に同じ型が来るように気をつける

黒魔術版 Record の定義

黒魔術版 Record の定義

```
type Record :: [(Symbol, Type)] → Type
newtype Record fs = MkRecord (Vector Any)

type Membership :: k → [k] → Type
newtype Membership l fs = Membership Int

getFieldOf :: Membership (l := v) fs → Record fs → v
getFieldOf (Membership i) (MkRecord xs) = unsafeCoerce (xs V.! i) :: v

updateOf :: Membership (l := v) fs → v → Record fs → Record fs
updateOf (Membership i) v (MkRecord xs) = MkRecord (xs V.// [(i, unsafeCoerce v)])
```


黒魔術版 Record の定義

```
type Record :: [(Symbol, Type)] → Type
newtype Record fs = MkRecord (Vector Any)

type Membership :: k → [k] → Type
newtype Membership l fs = Membership Int

getFieldOf :: Membership (l := v) fs → Record fs → v
getFieldOf (Membership i) (MkRecord xs) = unsafeCoerce (xs V.! i) :: v

updateOf :: Membership (l := v) fs → v → Record fs → Record fs
updateOf (Membership i) v (MkRecord xs) = MkRecord (xs V.// [(i, unsafeCoerce v)])
```

- **Record の内部表現は Vector Any に、Membership の表現も Int にし、再帰をやめる！**

黒魔術版 Record の定義

```
type Record :: [(Symbol, Type)] → Type
newtype Record fs = MkRecord (Vector Any)

type Membership :: k → [k] → Type
newtype Membership l fs = Membership Int

getFieldOf :: Membership (l := v) fs → Record fs → v
getFieldOf (Membership i) (MkRecord xs) = unsafeCoerce (xs V.! i) :: v

updateOf :: Membership (l := v) fs → v → Record fs → Record fs
updateOf (Membership i) v (MkRecord xs) = MkRecord (xs V.// [(i, unsafeCoerce v)])
```

- **Record の内部表現は Vector Any に、Membership の表現も Int にし、再帰をやめる！**
- 安全のため、Internal モジュール以外では**これらの内部構造は export しない**のが重要！

黒魔術版 Record の定義

```
type Record :: [(Symbol, Type)] → Type
newtype Record fs = MkRecord (Vector Any)

type Membership :: k → [k] → Type
newtype Membership l fs = Membership Int

getFieldOf :: Membership (l := v) fs → Record fs → v
getFieldOf (Membership i) (MkRecord xs) = unsafeCoerce (xs V.! i) :: v

updateOf :: Membership (l := v) fs → v → Record fs → Record fs
updateOf (Membership i) v (MkRecord xs) = MkRecord (xs V.// [(i, unsafeCoerce v)])
```

- **Record の内部表現は Vector Any に、Membership の表現も Int にし、再帰をやめる！**
- 安全のため、Internal モジュール以外では**これらの内部構造は export しない**のが重要！

🤔 $l \in xs$ の定式化はどうする？

$l \in xs$ の定式化：型レベルでインデックスを計算させる

- ・ GHC の型レベル自然数の計算機構を使おう！

$l \in xs$ の定式化：型レベルでインデックスを計算させる

- GHC の型レベル自然数の計算機構を使おう！

```
type IndexOf :: k → [k] → Nat
type family IndexOf x xs where
  IndexOf x '[] = TypeError (ShowType x :<: Text " not found!")
  IndexOf x (x ': xs) = 0
  IndexOf x (y ': xs) = 1 + IndexOf x xs

type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs

instance (KnownNat (IndexOf (l := Lookup' l fs) fs)) ⇒ l ∈ fs where
  membership = Membership $ fromIntegral $
    natVal $ Proxy @(IndexOf (l := Lookup' l fs) fs)
```


$l \in xs$ の定式化：型レベルでインデックスを計算させる

- GHC の型レベル自然数の計算機構を使おう！

要素のインデックスを得る
型レベル関数

```
type IndexOf :: k → [k] → Nat
type family IndexOf x xs where
  IndexOf x '[] = TypeError (ShowType x :<: Text " not found!")
  IndexOf x (x ': xs) = 0
  IndexOf x (y ': xs) = 1 + IndexOf x xs
```

```
type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs
```

```
instance (KnownNat (IndexOf (l := Lookup' l fs) fs)) ⇒ l ∈ fs where
  membership = Membership $ fromIntegral $
    natVal $ Proxy @(IndexOf (l := Lookup' l fs) fs)
```

$l \in xs$ の定式化：型レベルでインデックスを計算させる

- GHC の型レベル自然数の計算機構を使おう！

要素のインデックスを得る
型レベル関数

GHC の型レベル自然数
(GHC 9.2+: `Nat = Natural`)

```
type IndexOf :: k → [k] → Nat
type family IndexOf x xs where
  IndexOf x '[] = TypeError (ShowType x <> Text " not found!")
  IndexOf x (x ': xs) = 0
  IndexOf x (y ': xs) = 1 + IndexOf x xs
```

```
type (∈) :: Symbol → [(Symbol, Type)] → Constraint
class l ∈ fs where
  membership :: Membership (l := Lookup' l fs) fs
```

```
instance (KnownNat (IndexOf (l := Lookup' l fs) fs)) ⇒ l ∈ fs where
  membership = Membership $ fromIntegral $
    natVal $ Proxy @(IndexOf (l := Lookup' l fs) fs)
```

$l \in xs$ の定式化：型レベルでインデックスを計算させる

- GHC の型レベル自然数の計算機構を使おう！

要素のインデックスを得る
型レベル関数

GHC の型レベル自然数
(GHC 9.2+: `Nat = Natural`)

```
type IndexOf :: k → [k] → Nat
type family IndexOf x xs where
  IndexOf x '[] = TypeError (ShowType x <> Text " not found!")
  IndexOf x (x ': xs) = 0
  IndexOf x (y ': xs) = 1 + IndexOf x xs
```

```
type (∈) :: Symbol → [Symbol] → Constraint
class l ∈ fs where
  membership :: Membership l fs
```

KnownNat n
「コンパイル時に n の値が判っている」

```
instance (KnownNat (IndexOf (l := Lookup' l fs) fs)) ⇒ l ∈ fs where
  membership = Membership $ fromIntegral $
    natVal $ Proxy @(IndexOf (l := Lookup' l fs) fs)
```


$l \in xs$ の定式化：型レベルでインデックスを計算させる

- GHC の型レベル自然数の計算機構を使おう！

要素のインデックスを得る
型レベル関数

GHC の型レベル自然数
(GHC 9.2+: `Nat = Natural`)

```
type IndexOf :: k → [k] → Nat
type family IndexOf x xs where
  IndexOf x '[] = TypeError (ShowType x <> Text " not found!")
  IndexOf x (x ': xs) = 0
  IndexOf x (y ': xs) = 1 + IndexOf x xs
```

```
type (∈) :: Symbol → [Symbol] → Constraint
class l ∈ fs where
  membership :: Membership l fs
```

KnownNat n
「コンパイル時に n の値が判っている」

```
instance (KnownNat (IndexOf (l := Lookup' l fs) fs)) ⇒ l ∈ fs where
  membership = Membership $ fromIntegral $
    natVal $ Proxy @(IndexOf (l := Lookup' l fs) fs)
```

KnownNat n の値を取り出す

$l \notin fs$ と insert

$\text{l} \notin \text{fs}$ と `insert`

- $\notin$ は前回と同様でよい: `type l  $\notin$  fs = Lookup l fs ~ Nothing`

l ∉ fs と insert

- ∉ は前回と同様でよい： `type l ∉ fs = Lookup l fs ~ Nothing`
- 新規フィールドを足す insert は、内部的には単純に `cons + unsafeCoerce`

l ∉ fs と insert

- ∉ は前回と同様でよい : `type l ∉ fs = Lookup l fs ~ Nothing`
- 新規フィールドを足す insert は、内部的には単純に `cons + unsafeCoerce`

```
type l ∉ fs = Lookup l fs ~ 'Nothing
```

```
insert :: ∀ fs v. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

```
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```


l ∉ fs と insert

- ∉ は前回と同様でよい： `type l ∉ fs = Lookup l fs ~ Nothing`
- 新規フィールドを足す insert は、内部的には単純に `cons + unsafeCoerce`

```
type l ∉ fs = Lookup l fs ~ 'Nothing
```

```
insert :: ∀ fs v. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
```

```
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```

- これで全ての API を内部 Vector で置き換えられた！

l ∉ fs と insert

- ∉ は前回と同様でよい : `type l ∉ fs = Lookup l fs ~ Nothing`
- 新規フィールドを足す insert は、内部的には単純に `cons + unsafeCoerce`

```
type l ∉ fs = Lookup l fs ~ 'Nothing

insert :: ∀ fs v. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```

- これで全ての API を内部 Vector で置き換えられた！
- 正当性の保証はどうする？

l ∉ fs と insert

- ∉ は前回と同様でよい : `type l ∉ fs = Lookup l fs ~ Nothing`
- 新規フィールドを足す insert は、内部的には単純に `cons + unsafeCoerce`

```
type l ∉ fs = Lookup l fs ~ 'Nothing

insert :: ∀ fs v. ∀ l → (l ∉ fs) ⇒ v → Record fs → Record ((l := v) : fs)
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```

- これで全ての API を内部 Vector で置き換えられた！
- 正当性の保証はどうする？
 - テストを書こう！

unsafe 実装の正当性の保証

unsafe 実装の正当性の保証

- ・ **テスト** を書く

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる

unsafe 実装の正当性の保証

- テストを書く
 - Safe 実装が手に入る場合は、それを Ground Truth として比較させる
 - 直接期待される性質を書く（Property Base Testing!）

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く（**Property Base Testing!**）
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く（**Property Base Testing!**）
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る（実装例）

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く（**Property Base Testing!**）
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る（実装例）
 - 今日び**テストは LLM が書いてくれる**ので、ちゃんと指示してちゃんと中身を確認しましょう

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く（**Property Base Testing!**）
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る（実装例）
 - 今日び**テストは LLM が書いてくれる**ので、ちゃんと指示してちゃんと中身を確認しましょう
- **形式証明**したり**自動検証**させる（今回のをちゃんと記述するのは大変そう）

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く (**Property Base Testing!**)
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る (実装例)
 - 今日び**テストは LLM が書いてくれる**ので、ちゃんと指示してちゃんと中身を確認しましょう
- **形式証明**したり**自動検証**させる (今回のをちゃんと記述するのは大変そう)
 - Haskell なら Liquid Haskell で付加的な条件を検証したり、Agda から Extract するなど

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く (**Property Base Testing!**)
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る (実装例)
 - 今日び**テストは LLM が書いてくれる**ので、ちゃんと指示してちゃんと中身を確認しましょう
- **形式証明**したり**自動検証**させる (今回のをちゃんと記述するのは大変そう)
 - Haskell なら Liquid Haskell で付加的な条件を検証したり、Agda から Extract するなど
 - Rust なら MIRI, RustBelt, RustHorn, Verus, etc...

unsafe 実装の正当性の保証

- **テスト**を書く
 - Safe 実装が手に入る場合は、**それを Ground Truth として比較**させる
 - 直接**期待される性質**を書く (**Property Base Testing!**)
 - unsafe は境界でエラーが起きがちなので、**ランダムテスト**を書こう
 - Field の **Get / Set の命令列をランダムに生成**して挙動を見る (実装例)
 - 今日び**テストは LLM が書いてくれる**ので、ちゃんと指示してちゃんと中身を確認しましょう
- **形式証明**したり**自動検証**させる (今回のをちゃんと記述するのは大変そう)
 - Haskell なら Liquid Haskell で付加的な条件を検証したり、Agda から Extract するなど
 - Rust なら MIRI, RustBelt, RustHorn, Verus, etc...
 - Rust の unsafe についていえば、**完全ではないが unsafe {} ブロックをある程度自動検証してくれるツールがあるので「型が強くても unsafe 使ったら意味ない」は偽**と言える

Step 3: 線型型を用いた
更なる安全性と効率性の両立

とこるで

Unsafe 実装は
本当に速いのか？

ベンチマーク



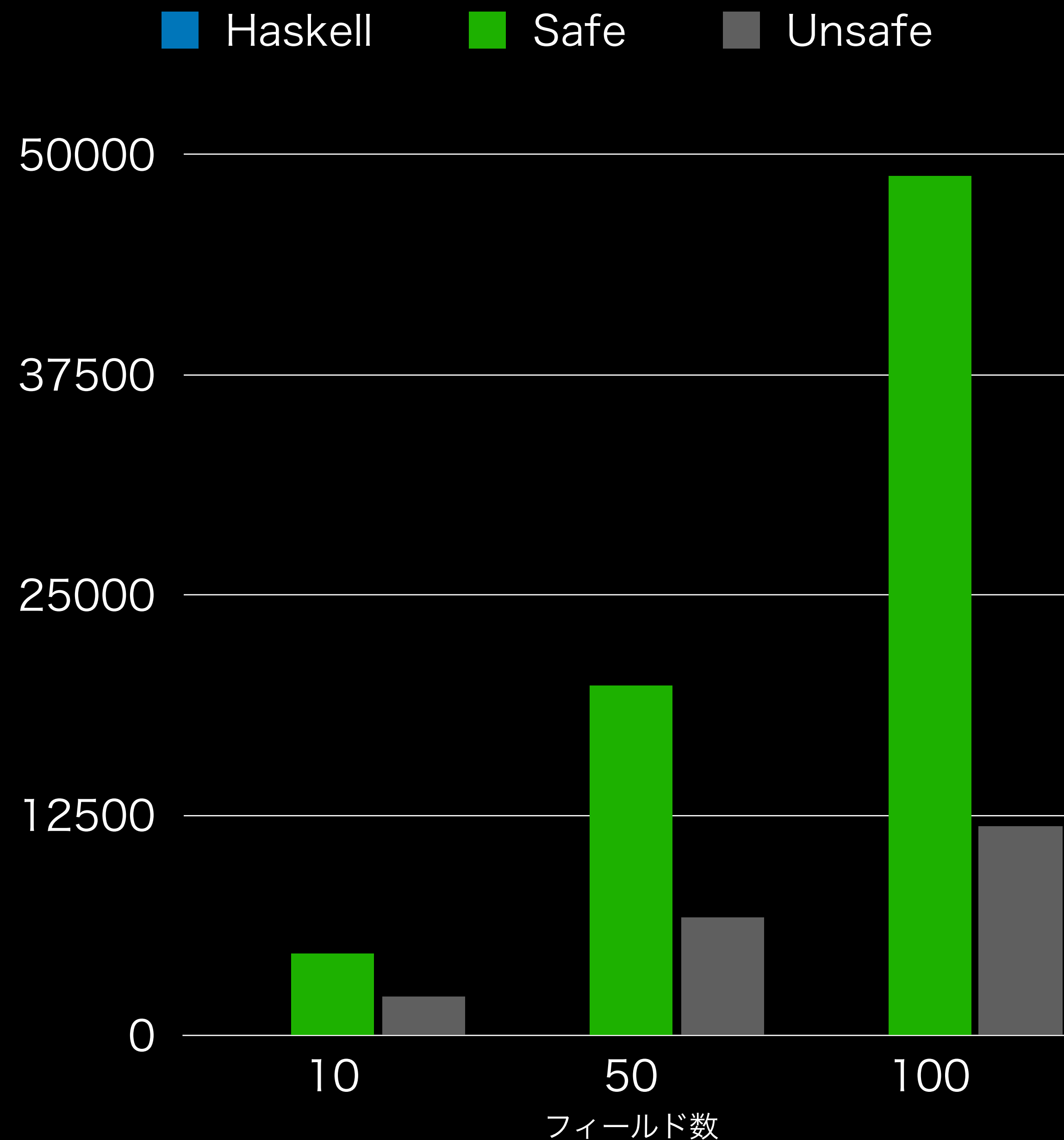
ベンチマーク

- ・ フィールド数10, 50, 100のレコードに対して、ランダムに生成した400個の get / update 列を実行
 - ・ Haskell: 普通のレコード
 - ・ Safe / Unsafe いままでの



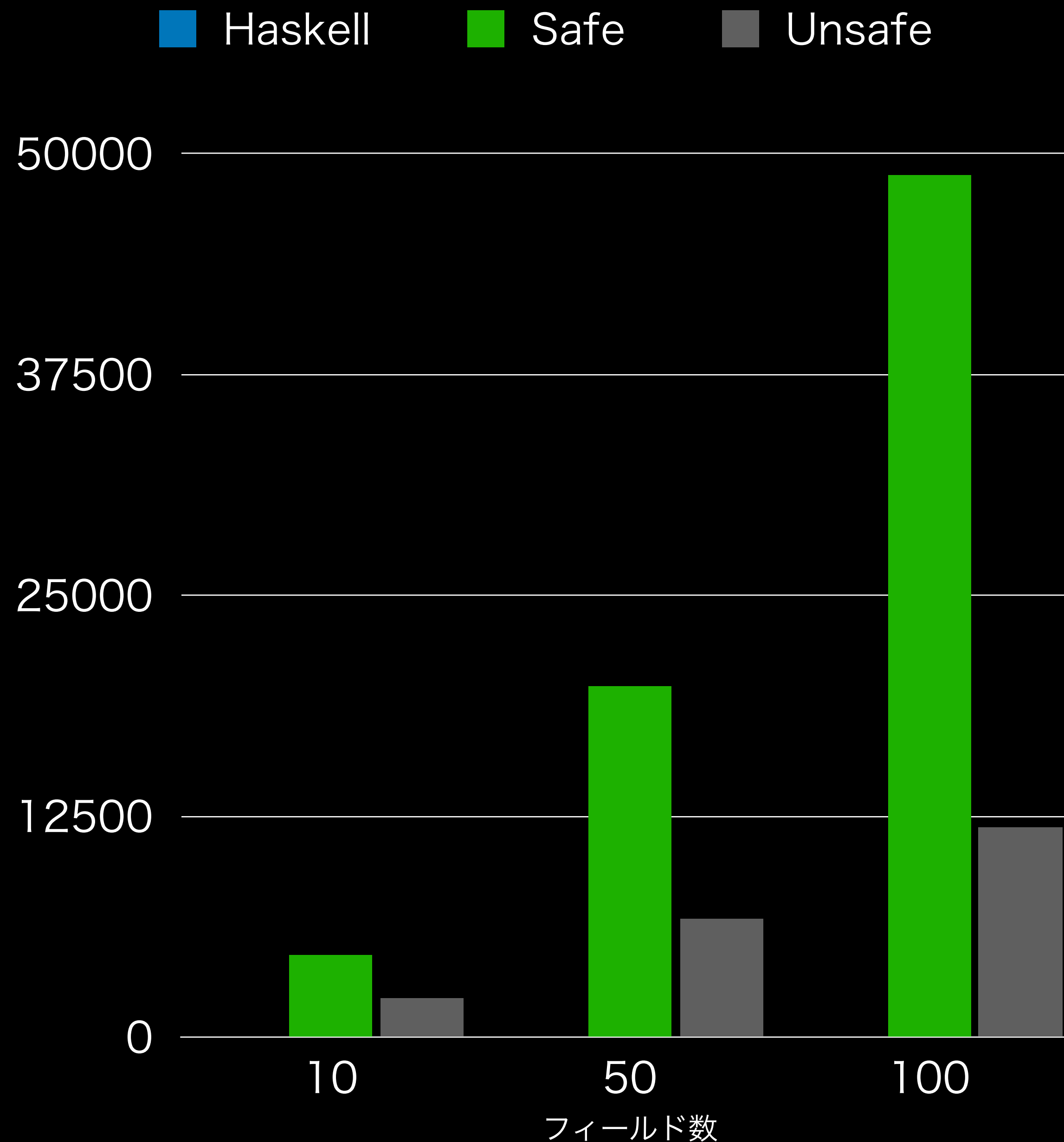
ベンチマーク

- ・ フィールド数10, 50, 100のレコードに対して、ランダムに生成した400個の get / update 列を実行
 - ・ Haskell: 普通のレコード
 - ・ Safe / Unsafe いままでの



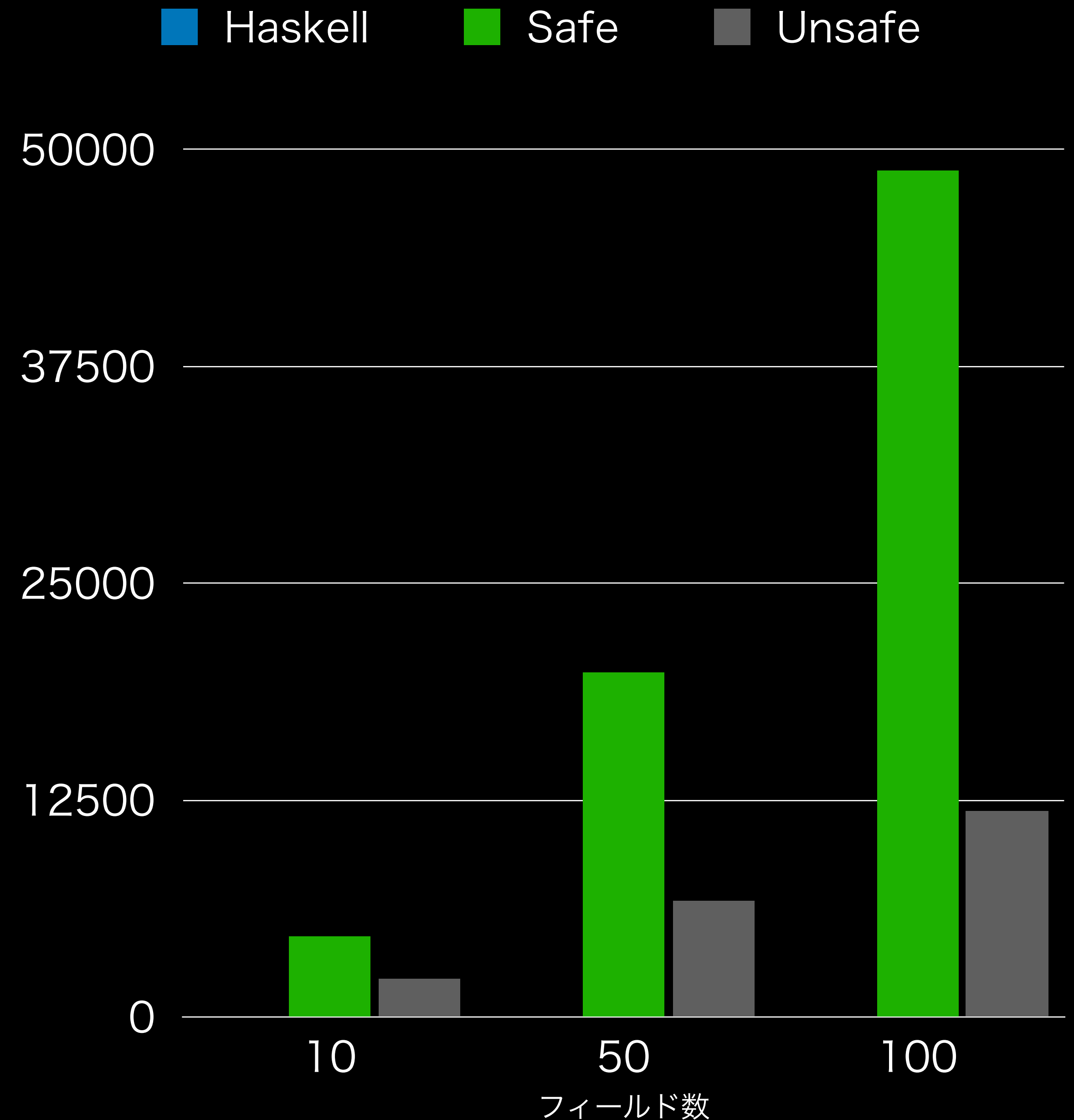
ベンチマーク

- ・ フィールド数10, 50, 100のレコードに対して、ランダムに生成した400個の get / update 列を実行
 - ・ Haskell: 普通のレコード
 - ・ Safe / Unsafe いままでの
- ・ 拡張性を考慮していない Haskell は当然速い（これに勝つことは目標じゃない）



ベンチマーク

- ・ フィールド数10, 50, 100のレコードに対して、ランダムに生成した400個の get / update 列を実行
 - ・ Haskell: 普通のレコード
 - ・ Safe / Unsafe いままでの
- ・ 拡張性を考慮していない Haskell は当然速い（これに勝つことは目標じゃない）
- ・ Safe よりも Unsafe ははるかにいいけど、それでも線型に悪化しているように見える？



実はまだパフォーマンスに改善の余地が
cons の問題

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

😓 ところが `update` や新規フィールドの追加操作 `insert` は $O(N)$ になっている！

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

😓 ところが `update` や新規フィールドの追加操作 `insert` は $O(N)$ になっている！

・ `insert`: `HList` ベース実装では **ただの連結リストの `cons` なので $O(1)$ だった**

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

😓 ところが `update` や新規フィールドの追加操作 `insert` は $O(N)$ になっている！

- ・ `insert`: `HList` ベース実装では **ただの連結リストの `cons` なので $O(1)$ だった**
- ・ **`Vector` の `cons` や `update` は再アロケートが発生し、最悪ケースで $O(N)$ に！**

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

😓 ところが `update` や新規フィールドの追加操作 `insert` は $O(N)$ になっている！

- ・ `insert`: `HList` ベース実装では **ただの連結リストの `cons` なので $O(1)$ だった**
- ・ **`Vector` の `cons` や `update` は再アロケートが発生し、最悪ケースで $O(N)$ に！**

```
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```

実はまだパフォーマンスに改善の余地が cons の問題

😊 ここまでの変更で `getField` については $O(1)$ になった

😓 ところが `update` や新規フィールドの追加操作 `insert` は $O(N)$ になっている！

- ・ `insert`: `HList` ベース実装ではただの連結リストの `cons` なので $O(1)$ だった
- ・ `Vector` の `cons` や `update` は再アロケートが発生し、最悪ケースで $O(N)$ に！

```
insert _ x (MkRecord fs) = MkRecord $ V.cons (unsafeCoerce x) fs
```

😇 `insert` を繰り返してレコードを構築しようとする最悪 $O(N^2)$ にかかってしまう！

可変配列を使えないか？

可変配列を使えないか？

- ・ 典型的には、レコードのフィールド集合は確定しているはず

可変配列を使えないか？

- 典型的には、レコードのフィールド集合は確定しているはず
- 予め必要な要素数の Vector を初期化して使い回せないか？

可変配列を使えないか？

- 典型的には、レコードの**フィールド集合は確定している**はず
- 予め**必要な要素数の Vector** を初期化して使い回せないか？
- 問題：使い回すには**可変配列**が必要！**IO や ST モナドが必要**に……

可変配列を使えないか？

- 典型的には、レコードの**フィールド集合は確定している**はず
- 予め**必要な要素数の Vector** を初期化して使い回せないか？
- 問題：使い回すには**可変配列**が必要！**IO や ST モナドが必要**に……
- なぜモナドが必要なのか？

可変配列を使えないか？

- 典型的には、レコードの**フィールド集合は確定している**はず
- 予め**必要な要素数の Vector** を初期化して使い回せないか？
- 問題：使い回すには**可変配列**が必要！**IO や ST モナドが必要**に……
- なぜモナドが必要なのか？
 - 可変配列を**複数箇所**で変更していると、順番によって結果が変わる！

可変配列を使えないか？

- 典型的には、レコードの**フィールド集合は確定している**はず
- 予め**必要な要素数の Vector** を初期化して使い回せないか？
- 問題：使い回すには**可変配列**が必要！**IO や ST モナドが必要**に……
- なぜモナドが必要なのか？
 - 可変配列を**複数箇所**で変更していると、順番によって結果が変わる！
 - IO モナドを使って副作用を明示するか、STを使って閉じ込める必要がある

可変配列を使えないか？

- 典型的には、レコードの**フィールド集合は確定している**はず
- 予め**必要な要素数の Vector** を初期化して使い回せないか？
- 問題：使い回すには**可変配列**が必要！**IO や ST モナドが必要**に……
- なぜモナドが必要なのか？
 - 可変配列を**複数箇所**で変更していると、順番によって結果が変わる！
 - IO モナドを使って副作用を明示するか、STを使って閉じ込める必要がある
- じゃあ、**配列が常に一箇所**でしか**所有されない**ようにすればいいのでは……？

線型型とは？

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \multimap b$ が使える

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \multimap b$ が使える
 - 「 $f :: a \multimap b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \multimap b$ が使える
 - 「 $f :: a \multimap b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \rightarrow b$ が使える
 - 「 $f :: a \rightarrow b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \multimap b$ が使える
 - 「 $f :: a \multimap b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！
 - `linear-base` パッケージの `Data.Array.Mutable.Linear` モジュールがそれ

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \rightarrow b$ が使える
 - 「 $f :: a \rightarrow b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！
 - linear-base パッケージの `Data.Array.Mutable.Linear` モジュールがそれ

```
alloc :: Int → a → (Array a → Ur b) → Ur b
set   :: Int → a → Array a → Array a
get   :: Int → Array a → (Ur a, Array a)
freeze :: Array a → Ur (Vector a)
```

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \rightarrow b$ が使える
 - 「 $f :: a \rightarrow b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！
- linear-base パッケージの `Data.Array.Mutable.Linear` モジュールがそれ

$\text{Ur } b$: 何回も使える b の意

```
alloc :: Int → a → (Array a →  $\text{Ur } b$ ) →  $\text{Ur } b$ 
set   :: Int → a → Array a → Array a
get   :: Int → Array a → ( $\text{Ur } a$ , Array a)
freeze :: Array a →  $\text{Ur } (\text{Vector } a)$ 
```


線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \rightarrow b$ が使える
 - 「 $f :: a \rightarrow b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！
- linear-base パッケージの `Data.Array.Mutable.Linear` モジュールがそれ

$\text{Ur } b$: 何回も使える b の意

```
alloc :: Int → a → (Array a →  $\text{Ur } b$ ) →  $\text{Ur } b$ 
set   :: Int → a → Array a → Array a
get   :: Int → Array a → ( $\text{Ur } a$ , Array a)
freeze :: Array a →  $\text{Ur } (\text{Vector } a)$ 
```

資源の線型性は関数の内側でしか担保されないので、継続渡しを使う

線型型とは？

- Linear Haskell の線型型：新しい**線型関数型** $a \multimap b$ が使える
 - 「 $f :: a \multimap b$ 」は適用式 $f\ x$ が「**ちょうど一回消費**」されたら、その内側で**引数 x も「ちょうど一回消費**」されることを保証してくれる
 - Rust の $\text{FnOnce}(A) \rightarrow B$ と似ている（但しRustは**高々一回**までしか保証しない）
- 線型型で**所有者が常に一人**であることを保証でき**純粹に可変配列が扱える**！
- linear-base パッケージの `Data.Array.Mutable.Linear` モジュールがそれ

$\text{Ur } b$: 何回も使える b の意

```
alloc :: Int → a → (Array a  $\multimap$   $\text{Ur } b$ )  $\multimap$   $\text{Ur } b$ 
set   :: Int → a → Array a  $\multimap$  Array a
get   :: Int → Array a  $\multimap$  ( $\text{Ur } a$ , Array a)
freeze :: Array a  $\multimap$   $\text{Ur } (\text{Vector } a)$ 
```

資源の線型性は関数の内側でしか担保されないので、継続渡しを使う

$O(1)$

線型型を使った Mutable Record

- アイデア：線型にしか扱えない可変な MRecord を導入 (M is for Mutable)

```
type    MRecord :: [(Symbol, k)] → [(Symbol, k)] → Type
newtype MRecord      absent      fs      = MRecord (LA.Array Any)

newRecord ::
  ∀ fs f. (KnownNat (Length fs)) ⇒
    (MRecord fs s %1 → MRecord '[] fs) %1 → Record fs
newRecord f =
  unur $ LA.alloc (fromIntegral $ natVal @(Length fs) Proxy) undefined
    \arr → Ur.lift (MkRecord . UnsafeHList) $
      LA.freeze (unMRecord $ f (MRecord arr))
modify :: Record xs → (MRecord '[] xs %1 → MRecord '[] xs) %1 → Record xs
```

- Mutable Array と同様、継続渡しで一意性を担保

線型型を使った Mutable Record

- アイデア：線型にしか扱えない可変な MRecord を導入 (M is for Mutable)

```
type    MRecord :: [(Symbol, k)] → [(Symbol, k)] → Type
newtype MRecord      absent      fs      = MRecord (LA.Array Any)

newRecord ::
  ∀ fs f. (KnownNat (Length fs)) ⇒
    (MRecord fs s %1 → MRecord '[] fs) %1 → Record fs
newRecord f =
  unur $ LA.alloc (fromIntegral $ natVal @(Length fs) Proxy) undefined
    \arr → Ur.lift (MkRecord . UnsafeHList) $
      LA.freeze (unMRecord $ f (MRecord arr))
modify :: Record xs → (MRecord '[] xs %1 → MRecord '[] xs) %1 → Record xs
```

最終的なフィールド集合

- Mutable Array と同様、継続渡しで一意性を担保

線型型を使った Mutable Record

- アイデア：線型にしか扱えない可変な MRecord を導入 (M is for Mutable)

```
type MRecord :: [(Symbol, k)] → Type
newtype MRecord absent fs = MRecord (LA.Array Any)

newRecord ::
  ∀ fs f. (KnownNat (Length fs)) ⇒
    (MRecord fs s %1 → MRecord '[] fs) %1 → Record fs
newRecord f =
  unur $ LA.alloc (fromIntegral $ natVal @(Length fs) Proxy) undefined
    \arr → Ur.lift (MkRecord . UnsafeHList) $
      LA.freeze (unMRecord $ f (MRecord arr))
modify :: Record xs → (MRecord '[] xs %1 → MRecord '[] xs) %1 → Record xs
```

最終的なフィールド集合

「未初期化」フィールド一覧

- Mutable Array と同様、継続渡しで一意性を担保

線型型を使った Mutable Record

- アイデア：線型にしか扱えない可変な MRecord を導入 (M is for Mutable)

```
type MRecord :: [(Symbol, k)] → [(Symbol, k)] → Type
newtype MRecord absent fs = MRecord (LA.Array Any)

newRecord ::
  ∀ fs f. (KnownNat (Length fs)) ⇒
    (MRecord fs s %1 → MRecord '[] fs) %1 → Record fs
newRecord f =
  unur $ LA.alloc (fromIntegral $ natVal @(Length fs) Proxy) undefined
    \arr → Ur.lift (MkRecord . UnsafeHList) $
    LA.freeze (unMRecord $ f (MRecord arr))
modify :: Record xs → (MRecord '[] xs %1 → MRecord '[] xs) %1 → Record xs
```

最終的なフィールド集合

「未初期化」フィールド一覧

初期化の完了を要求

- Mutable Array と同様、継続渡しで一意性を担保

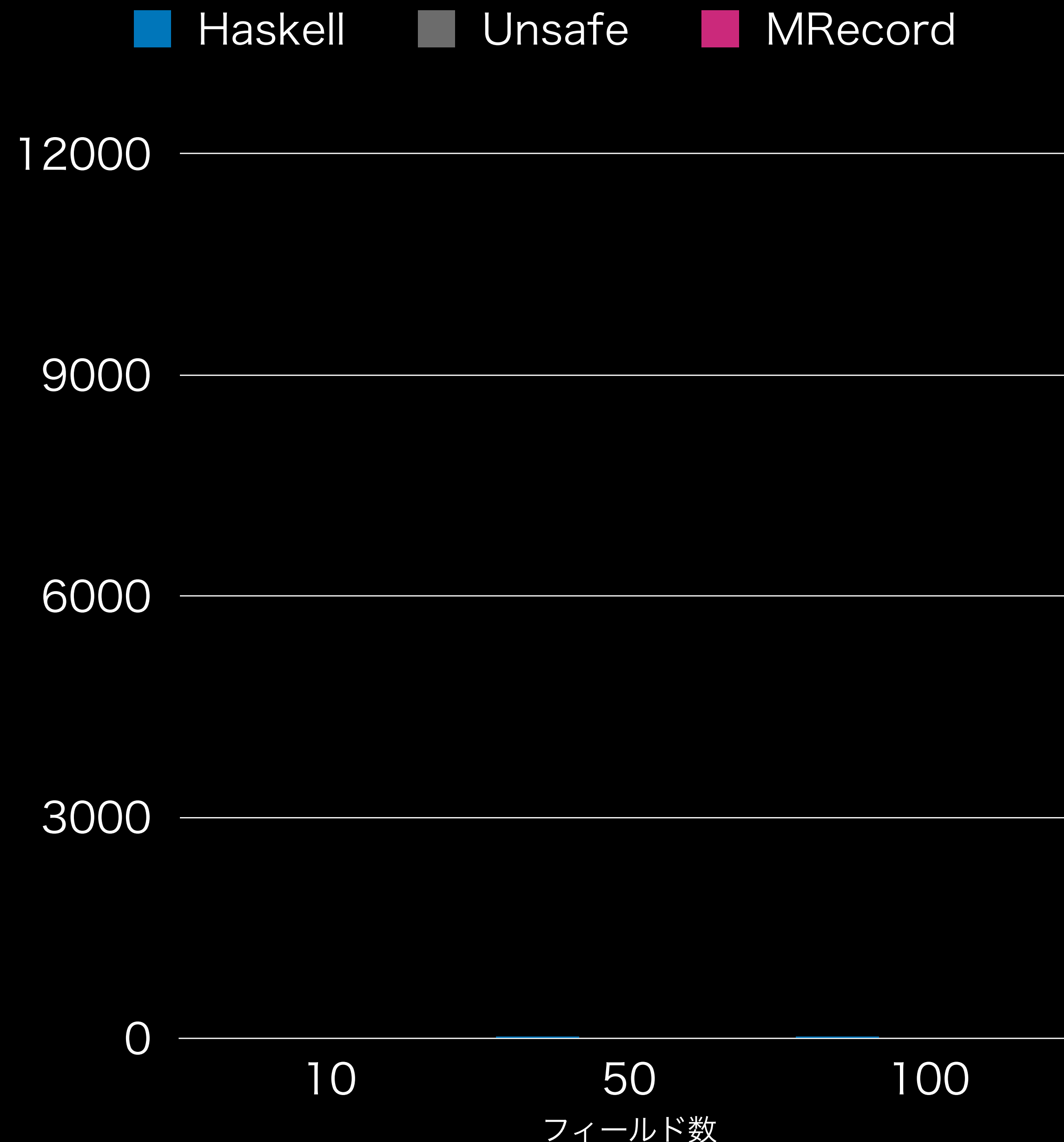
アクセス関数

```
set ::
  ∀ f absents fs. ∀ k → (k ∈ fs) ⇒
  Lookup' k fs →
  MRecord absents f fs %1 →
  MRecord (Delete (k := Lookup' k fs) absents) f fs
set k v (MRecord arr) =
  case membership @(k := Lookup' k fs) @fs of
    Membership i → MRecord $ LA.write arr i (unsafeCoerce v)

get ::
  ∀ fs absents f. ∀ k → (k ∈ fs, k ∉ absents) ⇒
  MRecord absents f fs %1 → (Ur (Lookup' k fs), MRecord absents f fs)
get k (MRecord arr) =
  case membership @'(k, v) @fs of
    Membership i → case LA.read arr i of
      (Ur elt, arr) → (Ur $ unsafeCoerce elt, MRecord arr)
```

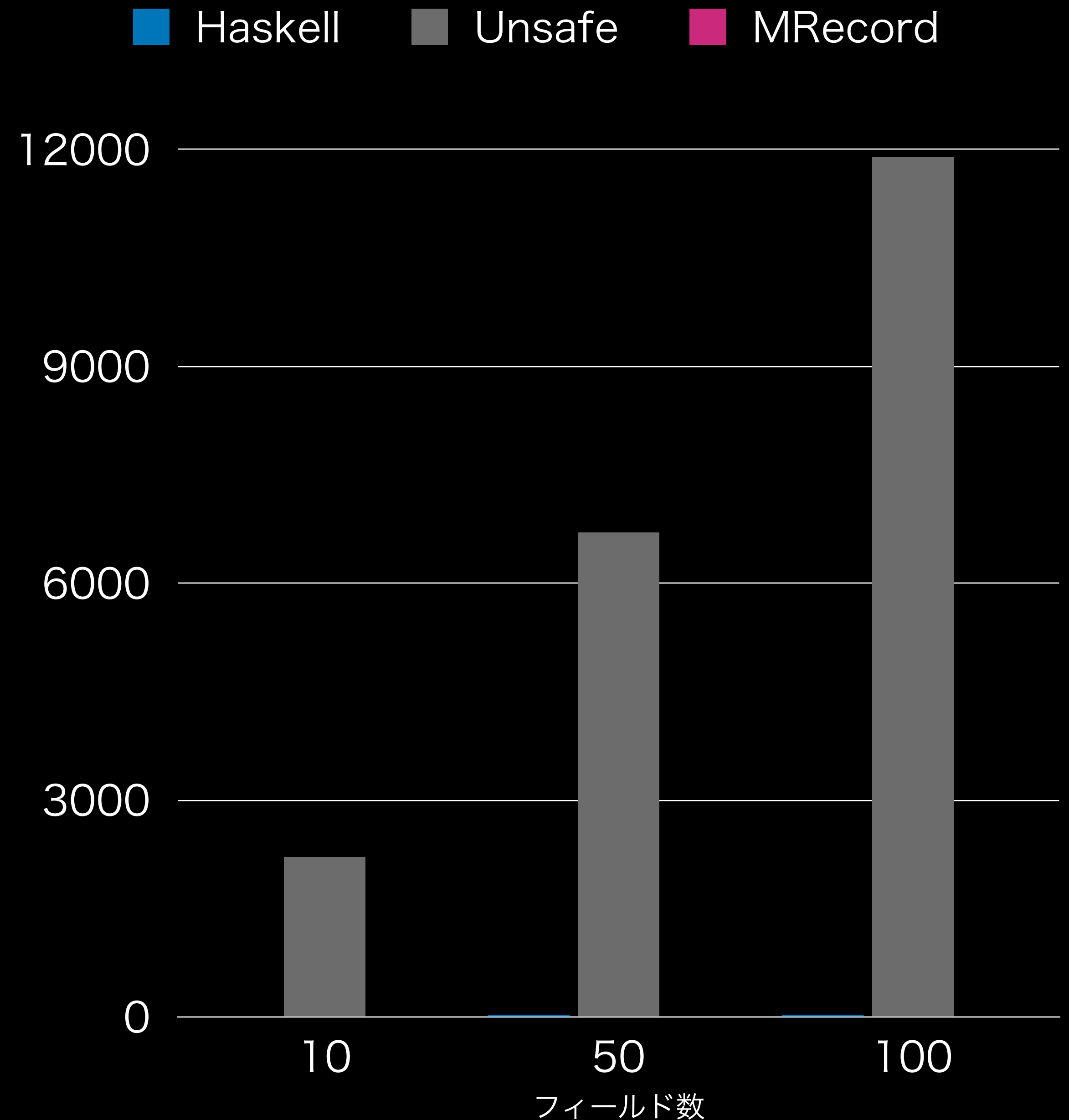

ベンチマーク・再訪

- 10, 50, 100 フィールドのレコードに対しランダムに生成した400個の get / update 列を実行
- MRecord を追加
 - 継続渡しの最初のアロケーションに掛かる時間は除く
- ほぼ定数になった！



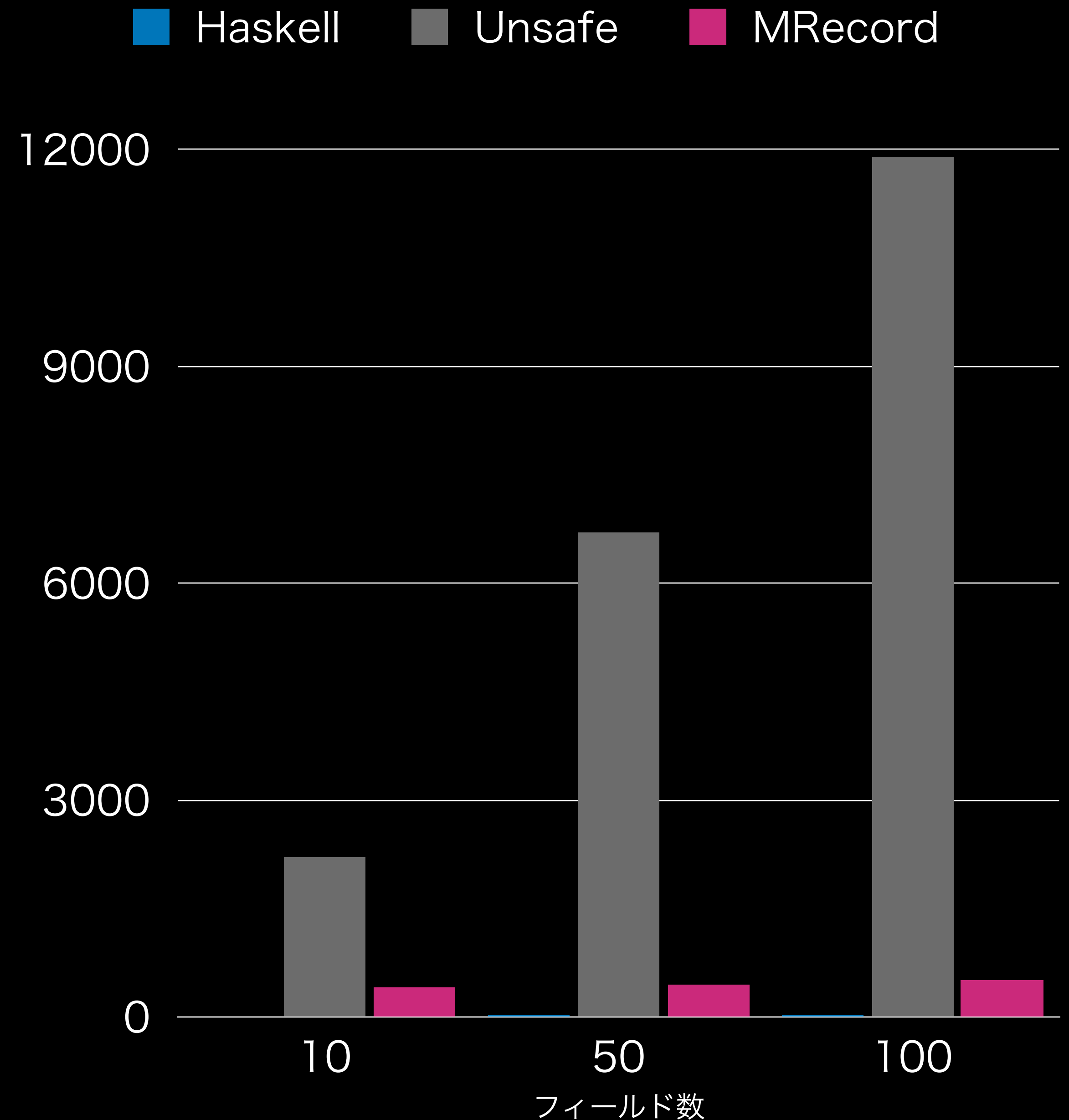
ベンチマーク・再訪

- 10, 50, 100 フィールドのレコードに対しランダムに生成した400個の get / update 列を実行
- MRecord を追加
 - 継続渡しの最初のアロケーションに掛かる時間は除く
- ほぼ定数になった！



ベンチマーク・再訪

- 10, 50, 100 フィールドのレコードに対しランダムに生成した400個の get / update 列を実行
- MRecord を追加
 - 継続渡しでの最初のアロケーションに掛かる時間は除く
- ほぼ定数になった！



とこるで

線型可変配列

ってどう

実装されてるの？

A.

unsafePerformIO

線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
    let new = unsafePerformIO (MV.replicate n x)
    in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
    Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
    let () = unsafePerformIO (MV.write arr i x)
    in Array arr
```


線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
  let new = unsafePerformIO (MV.replicate n x)
  in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
  Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
  let () = unsafePerformIO (MV.write arr i x)
  in Array arr
```

線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)
- **NOINLINE**: 純粹に偽装したIO処理が複数回実行されないようにインライン化を禁止している

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
  let new = unsafePerformIO (MV.replicate n x)
  in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
  Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
  let () = unsafePerformIO (MV.write arr i x)
  in Array arr
```


線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)
- **NOINLINE**: 純粋に偽装したIO処理が複数回実行されないようにインライン化を禁止している
- `Unsafe.toLinear :: (a → b) %1 → (a %1 → b)`

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
  let new = unsafePerformIO (MV.replicate n x)
  in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
  Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
  let () = unsafePerformIO (MV.write arr i x)
  in Array arr
```

線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)
- **NOINLINE**: 純粹に偽装したIO処理が複数回実行されないようにインライン化を禁止している
- `Unsafe.toLinear :: (a → b) %1 → (a %1 → b)`
 - 実際には任意に線型性を付け外しできる

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
  let new = unsafePerformIO (MV.replicate n x)
  in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
  Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
  let () = unsafePerformIO (MV.write arr i x)
  in Array arr
```

線型可変配列の実装 (簡略化版)

- `unsafePerformIO :: IO a → a` をフル活用 (実際にはもっとエグい奴)
- **NOINLINE**: 純粹に偽装したIO処理が複数回実行されないようにインライン化を禁止している
- `Unsafe.toLinear :: (a → b) %1 → (a %1 → b)`
 - 実際には任意に線型性を付け外しできる
- このように、表面的には安全な API を実際に安全に保つためには、かなり注意深い設計が必要 (**黒魔術が黒魔術たるゆえん**)

```
import Data.Vector.Mutable qualified as MV

data Array a = Array (MV.MVector RealWorld a)

alloc :: Int → a → (Array a %1 → Ur b) %1 → Ur b
{-# NOINLINE alloc #-}
alloc n x f =
  let new = unsafePerformIO (MV.replicate n x)
  in f (Array new)

get :: Int → Array a %1 → (Ur a, Array a)
{-# NOINLINE get #-}
get i = Unsafe.toLinear \(Array arr) → (
  Ur $ unsafePerformIO (MV.read arr i), Array arr)

set :: Int → a → Array a %1 → Array a
{-# NOINLINE set #-}
set i x = Unsafe.toLinear \(Array arr) →
  let () = unsafePerformIO (MV.write arr i x)
  in Array arr
```


まとめ：線型型による純粋性と破壊的変更の両立

unsafe 大車輪

- ・ 引数をきっかり一回使う関数型 $a \rightarrow b$ を使って、Haskell でも**所有権の一意性を保証**できる
 - ・ 拡張可能レコードへの応用：update が $O(1)$ の**可変レコード**
 - ・ 内部では線型型による**純粋な可変配列**を利用
- ・ 純粋な**線型可変配列の内側**では `unsafePerformIO` が大活躍
 - ・ 副作用が表に出ないようにインライン化の抑制が必要（黒魔術には責任が伴う）
 - ・ 逆に、**unsafePerformIO がなければ線型可変配列のような応用例も実装できない！**

補足と議論とまとめ

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

- Unsafe = **型システムを拡張するためのエスケープハッチ**である

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

- Unsafe = **型システムを拡張するためのエスケープハッチ**である
- Unsafe = 本来型システムが保証すべき性質を**実装者が代わりに保証します**という責任の表明

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

- Unsafe = **型システムを拡張するためのエスケープハッチ**である
- Unsafe = 本来型システムが保証すべき性質を**実装者が代わりに保証します**という責任の表明
- 強力な型システムと unsafe は互いに**補い合う関係**にある

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

- Unsafe = **型システムを拡張するためのエスケープハッチ**である
- Unsafe = 本来型システムが保証すべき性質を**実装者が代わりに保証します**という責任の表明
- 強力な型システムと unsafe は互いに**補い合う関係**にある
 - 本講演：拡張可能レコードを例に、**非効率な実装をGADTsで与え**、後から同じAPI を unsafeCoerce が必要だが効率的な配列実装を与えた

Unsafe は強力な型システム下でこそ威力を発揮

Unsafe = 型システムを拡張するエスケープハッチ

- Unsafe = **型システムを拡張するためのエスケープハッチ**である
- Unsafe = 本来型システムが保証すべき性質を**実装者が代わりに保証します**という責任の表明
- 強力な型システムと unsafe は互いに**補い合う関係**にある
 - 本講演：拡張可能レコードを例に、**非効率な実装をGADTsで与え**、後から同じAPI を unsafeCoerce が必要だが効率的な配列実装を与えた
 - 更に、線型型を使って効率的な可変レコードの実装も

unsafe にまつわる迷信（再掲）

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても **unsafe があっちゃ意味ないじゃん**

😡 **unsafe は罪！** unsafe 追放！

👻 unsafe がエルフの村を焼いたらしい

😞 unsafe はうちの裏には要らない

迷信に対する論駁

😓unsafe は「キケン」なので良くない

- ・「キケン」な関数は **unsafe 以外にもある**。何がいつ、どういう尺度で「キケン」なのかの意識が大事

😄いくら「強力な型システム」があっても **unsafe があっちゃ意味ない**じゃん

- ・強力な型システムがあるからこそ、unsafe によってユーザが拡張できる
 - ・そもそもマトモな型システムがなければ safe も unsafe もクソもない
- ・何かバグがあったら、まず unsafe の利用を疑うという指針にもなる
- ・Rust などには unsafe ブロックのコードを検証してくれるツールもある

😡**unsafe は罪**！unsafe 追放！

- ・unsafe は**実装者が原罪を負う**ことと引き換えに**強力なライブラリを提供する**という意思表示
- ・大いなる力は大いなる責任（検証責任）とともに

まとめ

- ・ 危険にもいろいろ：**正当性**（プログラムの正しさ）／**健全性**（型安全性）
- ・ unsafe は**強力な型システムの存在下でこそ輝く**
 - ・ そもそも**何も保証されていない型システムで安全性を考えてもしょうがない**
 - ・ 型システムの拡張機能はしばしば**裏で unsafe が使われることが前提**
- ・ unsafe は「型システムに代わって**実装者が安全性の保証**を引き受けます」という**原罪の表明**である
 - ・ もちろん、unsafe な機能の利用には**潜在的なリスク**が伴う
 - ・ API 自体の健全性は**非効率な代替実装を試みる**ことで示せることがある（**定義による拡張**）
 - ・ それが無理であっても、**テストや証明、自動検証器の活用**によって**罪を償却**し一歩上のプログラマへ
- ・ 型レベルプログラミングは必要な性質の「証拠」を具体化して考えると吉（BHK解釈インスパイア）

御清聴

ありがとう

ございました

Any Questions?

- ・ 危険にもいろいろ：**正当性**（プログラムの正しさ）／**健全性**（型安全性）
- ・ unsafe は**強力な型システムの存在下でこそ輝く**
 - ・ そもそも**何も保証されていない型システムで安全性を考えてもしょうがない**
 - ・ 型システムの拡張機能はしばしば**裏で unsafe が使われることが前提**
- ・ unsafe は「型システムに代わって**実装者が安全性の保証**を引き受けます」という**原罪の表明**である
 - ・ もちろん、unsafe な機能の利用には**潜在的なリスク**が伴う
 - ・ API 自体の健全性は**非効率な代替実装を試みる**ことで示せることがある（**定義による拡張**）
 - ・ それが無理であっても、**テストや証明、自動検証器の活用**によって**罪を償却**し一歩上のプログラマへ
- ・ 型レベルプログラミングは必要な性質の「証拠」を具体化して考えると吉（BHK解釈インスパイア）

おまけ1 : Show / Eq の実装

ところで……

Show 実装したくないですか？

- レコードを文字列表現に変換する Show のインスタンスが欲しい
- Record fs に対してどういう制約があればよさそうか？
 1. 各ラベルの「名前」の情報が欲しい
 2. 各フィールド値の Show インスタンス
- これを素直に書くと……

Show インスタンス 直書き

```
class ShowableRecord fs where
  showRecordFields :: Record fs → [String]

instance ShowableRecord '[] where showRecordFields NilR = []

instance
  (KnownSymbol l, Show v, ShowableRecord fs) ⇒
  ShowableRecord (l := v : fs)
  where
    showRecordFields (v :=> fs) =
      let fld = symbolVal (Proxy @l)
      in (fld < " = " < show v) : showRecordFields fs

instance (ShowableRecord fs) ⇒ Show (Record fs) where
  show r = "Record {" < (intercalate ", " $ showRecordFields r) < "}"
```

l の文字列表現がわかる、という制約

l の文字列表現を取得

- Eq が欲しかったらどうする？

Eq インスタンス 直書き

```
instance Eq (Record '[]) where  
  NilR == NilR = True
```

```
instance (Eq v, Eq (Record fs))  
  => Eq (Record (l := v : fs)) where  
  (v1 :> fs1) == (v2 :> fs2) = v1 == v2 && fs1 == fs2
```

パターニン

見えてきたよ！

各ワールドに対し～

という制約

+ 畳み込み

があればよさそう

All 制約：「各制約について～」を表す

```
type All :: (k → Constraint) → [k] → Constraint
type family All c xs where
  All c '[] = ()
  All c (x : xs) = (c x, All c xs)

class (KnownSymbol (Fst kv), Show (Snd kv))
  ⇒ ShowField kv
instance (KnownSymbol k, Show v) ⇒ ShowField (k := v)
```

😓 **問題**： All ShowField fs だけから各フィールドの KnownSymbol / Show を復元できない！

・ これは **Lookup' l fs ~ v だけから getField / update に必要な情報が取れなかったのと同根**

💡 今回も All の「証拠」を具体的に作ったらどうか？

証拠付きの All 制約

```
type All :: (k → Constraint) → [k] → Constraint
type family All c xs where
  All c '[] = ()
  All c (x : xs) = (c x, All c xs)

class (KnownSymbol (Fst kv), Show (Snd kv))
  ⇒ ShowField kv
instance (KnownSymbol k, Show v) ⇒ ShowField (k := v)
```

😓 **問題**： All ShowField fs だけから各フィールドの KnownSymbol / Show を復元できない！

・ これは **Lookup' l fs ~ v だけから getField / update に必要な情報が取れなかったのと同根**

💡 今回も All の「証拠」を具体的に作ったらどうか？

証拠つきの All 制約

- GADTs は型クラスの証拠 = 辞書を暗黙に保持できる (Dic1)
- Dic1 にパターンマッチすると、文脈内に型制約が入る
- あとは型リストの形に沿ってその「辞書」を集めよう: AllDic
- All は「**証拠となるAllの辞書がある**」という形で定式化する

```
data Dic1 c a where
  Dic1 :: (c a) => Dic1 c a
```

```
data AllDic c xs where
  DNil :: AllDic c '[]
  DCons :: Dict1 c x
        -> AllDic c xs
        -> AllDic c (x : xs)
```

```
class All c xs where
  allDict :: AllDic c xs
```

```
instance All c '[] where
  allDict = DNil
```

```
instance (c x, All c xs)
  => All c (x : xs) where
  allDict = DCons Dict1 allDict
```

Eq を書き直してみる

- AllDict と Record 二つを並べて zipWith できれば良さそう
- OverKV c (k := v) = c k v
- Right c l r = c l
- 演習問題：Show も All ベースで実装してみよう！

```
zipWithC ::
  ∀ c xs a. (All (OverKV c) xs) ⇒
  (∀ x. ∀ l → (c l x) ⇒ x → x → a) →
  Record xs → Record xs → [a]
zipWithC f r1 r2 = go allDict r1 r2
  where
    go :: AllDict c k v → Record xs → [a]
    go DNil NilR NilR = []
    go (DCons Dict1 ds) (v1 :> fs1)
      (v2 :> fs2 :: Record (kv : ws))
      = f (Fst kv) v1 v2 : go ds fs1 fs2

instance (All (OverKV (RightC Eq)) fs)
  ⇒ Eq (Record fs) where
  r1 == r2 = and $
    zipWithC @(RightC Eq) (\_ v1 v2 → v1 == v2) r1 r2
```

ところで……

- ・ 既視感ありませんか？

```
type Dicts
  :: (k → Constraint) → [k] → Type
data AllDic c xs where
  DNil  :: AllDic c '[]
  DCons :: Dict1 c x
        → AllDic c xs
        → AllDic c (x : xs)
```

- ・ HList と似てる！
 - ・ HList は「Type」のリストなのに対し多相的な k のリストを取る
- ・ これ統一できない？

ところで……

- ・ 既視感ありませんか？

```
type Dicts
  :: (k → Constraint) → [k] → Type
data AllDic c xs where
  DNil    :: AllDic c '[]
  DCons   :: Dict1 c x
           → AllDic c xs
           → AllDic c (x : xs)
```

```
type HList
  :: [Type] → Type
data HList xs where
  HNil    :: HList '[]
  (:-)    :: x
           → HList xs
           → HList (x : xs)
```

- ・ HList と似てる！
 - ・ HList は「Type」のリストなのに対し多相的な k のリストを取る
- ・ これ統一できない？

ところで……

- 既視感ありませんか？

```
type Dicts
  :: (k → Constraint) → [k] → Type
data AllDic c xs where
  DNil    :: AllDic c '[]
  DCons   :: Dict1 c x
           → AllDic c xs
           → AllDic c (x : xs)
```

```
type HList
  :: [Type] → Type
data HList xs where
  HNil    :: HList '[]
  (:-)    :: x
           → HList xs
           → HList (x : xs)
```

- HList

- HList

- これ続

```
type HList :: (k → Type) → [k] → Type
data HList f xs where
  HNil    :: HList f '[]
  (:-)    :: f x → HList f xs → HList f (x : xs)
```

高階 HList

```
type HList :: (k → Type) → [k] → Type
data HList f xs where
  HNil :: HList f '[]
  (:-) :: f x → HList f xs → HList f (x : xs)
```

- 新たに追加されたパラメータ $f :: k \rightarrow \text{Type}$ が色々なパターンを吸収してくれる
 - (Old) $\text{HList } xs \simeq \text{HList Identity } xs$ ($\text{newtype Identity } a = \text{Identity } a$)
 - $\text{AllDict } c \text{ } xs \simeq \text{HList (Dic1 } c) \text{ } xs$
- **Record 自体も高階HList と同値 !**
 - $\text{Record } xs \simeq \text{HList Field } xs$ ($\text{newtype Field } kv = \text{MkField (Snd } kv)$)